

TEAM ONBOARDING · 인수인계 문서

AI-V2X 스마트 지팡이 시스템 인수인계 설명서

신규 팀원을 위한 전체 시스템 안내서

우리가 무엇을 만드는가 · 9단계 구조 · RSU가 하는 일 · 서버/SUMO 연동

AI 기반 V2X 협력형 지능형 시각장애인 보행 안전 지팡이 시스템

본 문서는 저장소 실제 코드(arduino / lux / scripts / AI_Model)를 근거로 작성되었습니다.

문서 1 / 2 · 시스템편

처음 온 팀원이라면, 여기부터

이 시스템은 3개의 하드웨어 노드 + 1개의 판단 컴퓨터(Jetson) + 데이터/AI 파이프라인으로 이루어져 있습니다. 문서는 큰 그림 → 세부 순서로 읽으면 됩니다.

1. 큰 그림 (3~6p)

무엇을 만드는가, 3개 노드가 어떻게 연결되는가, 데이터가 어디로 흐르는가.

2. RSU 집중 (9~14p)

RSU 노드(ESP32 중계 + Jetson 두뇌)가 무엇을 받아 어떻게 판정하는지 단계별로.

3. 판정 위치·서버·SUMO (15~23p)

risk를 어디서 계산·max·레벨화하는지 (15·16p), 서버 연동, SUMO → AI 학습 흐름.

이 문서의 약속

실제 저장소 코드에서 확인한 **사실**만 적었습니다. "이렇게 하면 좋다"가 아니라 "지금 코드가 이렇게 되어 있다"입니다. 아직 정리 중이거나 버전이 어긋난 부분은 **주황색 주의 카드**로 표시했습니다.

딱 문서 안내

문서 2·코드 설명서에서는 파이썬 파일 하나하나가 무슨 일을 하는지 순서대로 설명합니다. 이 문서(시스템편)로 큰 그림을 잡은 뒤 문서 2로 코드를 보면 됩니다.

"보기 전에" 위험을 알린다

골목·주차장 출입구처럼 시야가 막힌 사각지대에서, 다가오는 차량을 시각장애인 보행자가 인지하기 전에 지팡이 진동·부저로 미리 경고하는 시스템입니다.

문제

흰지팡이·초음파 지팡이는 **사용자 앞·근거리**만 감지합니다. 모퉁이 뒤에서 접근하는 차량은 원리상 잡지 못합니다.

우리의 해법 — V2X 협력

차량·보행자·도로변 장치(RSU)가 **서로 위치·속도를 무선으로 교환**합니다. 개인 센서의 한계를 인프라 협력으로 넘습니다.

핵심 작동 3단계

1

감지

차량 위치를 실시간으로 받는다

>

2

판단

거리·속도·방향으로 위험도 계산

>

3

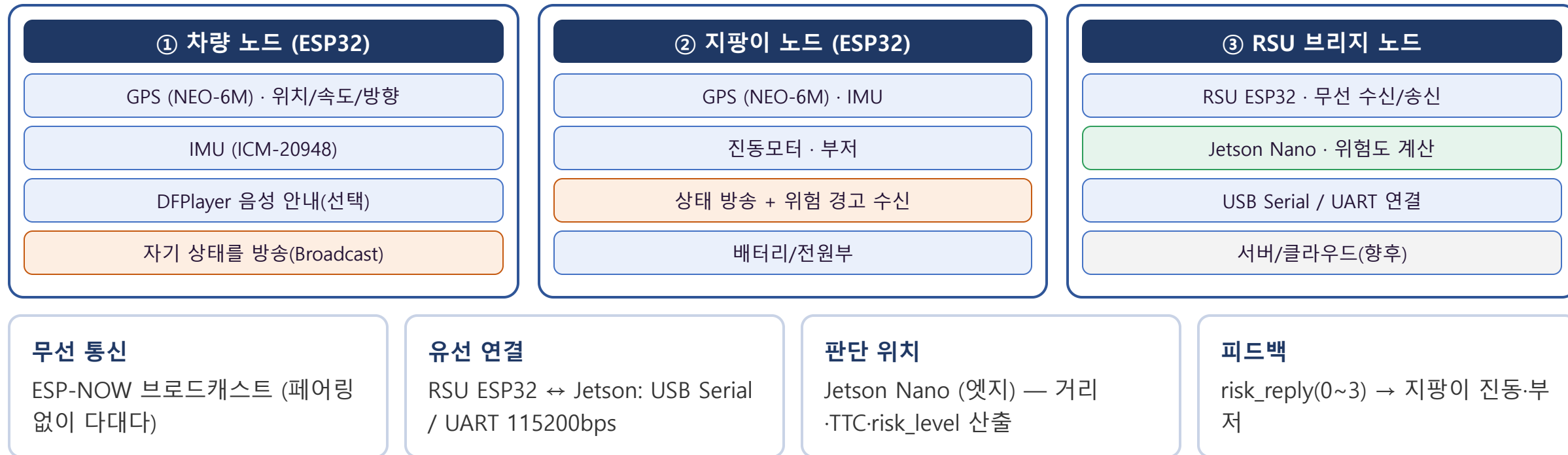
경보

지팡이 진동·부저로 알림

설계 철학 — 안전 우선

Fire-first / Confirm-later: 확신이 조금 낮아도 **일단 경고**하고 (지팡이), 정밀 판단으로 뒤에서 확정·해제합니다(RSU/Jetson). 놓치는 것보다 한 번 더 울리는 쪽을 택합니다.

세 개의 노드가 **ESP-NOW** 무선으로 상태를 교환하고, RSU가 **Jetson**에 넘겨 위험도를 계산합니다.



용어 **RSU**(Road-Side Unit) = 도로변 기지. **V2X** = 차량-사물 간 통신. **TTC**(Time To Collision) = 충돌까지 남은 예상 시간. **ESP-NOW** = ESP32끼리 쓰는 초경량 무선 프로토콜.

겹치는 게 아니라 이어진다 (정의 → 실행)

최민서와 내 파트가 겹쳐 보이지만, 실제로는 "무엇을 위험이라 부를지 정의"(최민서·오프라인) → "지금 이 순간 위험한지 실시간 판정"(나·온라인)으로 이어지는 구조입니다. (근거: integration_contract.md)

최민서 — 정의(오프라인)

SUMO 시뮬레이션 · zone 정의 · **risk 기준(점수표) 정의** · Transformer 학습→ONNX · 위험 지도 시각화

= 기준·라벨·모델을 만든다

나 — 실행(온라인) ★

ESP-NOW 수신 · RSU 브리지 · lux 파이프라인 · **실시간 계산·max 융합·risk 레벨 결정·전송** · ONNX 추론 실행

= 그 기준·모델로 실시간 판정한다

박중선 — 하드웨어

지팡이·차량 노드 배선, GPS/IMU/부저/진동모터, 전원·배터리, 시제품 조립

핵심 — risk를 실제로 "결정"하는 건 내 파트다

최민서는 "위험의 기준과 AI 모델"을 만들고, 나는 그것을 받아 **현장 데이터로 실시간 risk 계산 → 여러 판단을 max로 융합 → 최종 risk 0~3 결정 → 전송**합니다. 계산·판정·레벨화·max는 모두 내 쪽(Jetson)에서 일어납니다. (자세한 위치는 15p)

⚠ 겹쳐 보이는 진짜 이유 — 점수표가 두 곳에 복사됨

같은 점수표가 scripts/risk_calculator.py(최민서·라벨용 원본)와 lux/risk_scoring.py(내 실시간 복사본)에 **이중으로** 존재합니다. 정의는 하나(최민서 소유)여야 하고 실행만 두 곳이어야 합니다 → **단일 소스로 관리**해 어긋남(drift)을 막아야 합니다.

데이터는 이렇게 한 바퀴 돈다

지팡이·차량이 상태를 방송하면 → RSU가 모아 Jetson에 넘기고 → Jetson이 위험도를 계산해 → RSU를 거쳐 지팡이로 경고가 돌아옵니다.



두 개의 안전 경로 (이중화)

경로 A · 직접 통신: 차량↔지팡이가 RSU 없이 직접 주고받아 1차 즉시 경고. (구현·검증 완료)

경로 B · RSU 경유: RSU→Jetson이 정밀 판단해 **확정·보강**. 한쪽이 놓쳐도 다른 쪽이 잡습니다.

지금 무엇이 실제로 도는가

현재 검증된 주력 경로는 **RSU(ESP32) → Jetson(lux/ 파이프라인) → 지팡이**이며, 위험도는 **거리+TTC** 규칙으로 즉시 산출됩니다. AI(Transformer) 예측과 서버 연동은 **확장 단계**입니다 (17·20p).

지금까지의 단계 (arduino/ 01~10)

각 폴더는 서로 다른 ESP32에 업로드하는 펌웨어입니다. 번호는 개발 순서이며, 굵게 표시한 것이 현재 데모의 주력본입니다.

#	업로드 대상	역할	통신
01	지팡이 송신 ESP32	GPS-IMU 수집 후 수신기로 송신 (초기 버전)	ESP-NOW(1:1)
02	수신 ESP32	수신 → Jetson UART 전달 + 진동/부저	ESP-NOW + UART
03	피드백 ESP32	risk 명령을 진동·부저로 출력 (독립 테스트)	Wi-Fi UDP:6001
04	차량 ESP32	차량 상태 송신 + 응답 수신 + 음성 (초기)	ESP-NOW(1:1)
05	지팡이 ESP32	차량 수신 → 거리 계산 → Jetson 브리지	ESP-NOW 방송 + UART
06	차량 ESP32 ★	상태 방송 + 거리/TTC 위험도 계산 → 지팡이 직접 경고	ESP-NOW 방송/지정
07	차량 ESP32	Jetson UART 흐름 단독 점검 (現 미사용)	ESP-NOW + UART
08	RSU ESP32	차량 전용 무선↔Jetson USB 브리지	ESP-NOW + USB Serial
09	지팡이 ESP32 ★	상태 방송 + 위험 경고 수신·출력	ESP-NOW 방송
10	RSU ESP32 ★	차량+지팡이 모두 수신→Jetson, risk 중계 (실사용 RSU)	ESP-NOW + USB Serial

읽는 법 실제 데모 주력 = **06(차량) · 09(지팡이) · 10(RSU)**. 08은 10의 단일(차량 전용) 축소판. 01~04는 초기 프로토콜 버전(다음 페이지 참고).

"프로토콜 두 세대"를 먼저 이해하기

신규 팀원이 가장 헷갈리는 부분입니다. 겉보기엔 하나의 시스템이지만, 무선 패킷 구조가 다른 두 계열이 폴더에 섞여 있습니다.

계열 A — 초기 "직접 페어링"

스케치 01 · 02 · 04

- 구조체 v2x_message_t (IMU 6축 포함)
- 특정 MAC 주소를 코드에 하드코딩해 1:1 통신
- magic/version 없음

계열 B — V2X 브로드캐스트 ★현행

스케치 05 · 06 · 07 · 08 · 09 · 10

- 구조체 v2x_status_message_t (magic "v2x1", node_id)
- 페어링 없이 방송(다대다)
- 메시지 타입: 차량상태 / 지팡이상태 / 위험경고 / RSU응답

⚠ 인수인계 주의 — 계열 B 안에도 버전 차이가 있다

차량 06·지팡이 09는 **version 2**(위험 메시지에 distance_m·closing_speed·ttc 포함)인데, RSU 08·10은 **version 1**입니다. 지팡이 09는 version이 2가 아니면 패킷을 버리므로, **현재 코드 그대로는 "10 RSU → 09 지팡이" 경고 전달이 실패**합니다. RSU를 실제로 쓰려면 10을 version 2 + distance 필드로 올려야 합니다. (그래서 지금 데모의 직접 경고는 06 ↔ 09 직접 통신으로 동작합니다.)

한 줄 요약 01~04(계열 A)는 초기 실험, 05~10(계열 B)가 현행. 그중 06/09는 v2, 08/10 RSU는 v1 — 이 버전만 맞추면 RSU 경유 경로가 완성됩니다.

RSU 노드 = 우편함(ESP32) + 두뇌(Jetson)

"RSU"라는 한 단어에 **두 개의 장치**가 들어 있습니다. 이 둘을 구분하지 않으면 "RSU가 계산하나?"에서 계속 헷갈립니다. **계산·AI·판정**은 모두 **RSU 노드** 안의 **Jetson**에서 일어납니다.

① RSU ESP32 — 우편함 (중계만)

무선으로 온 상태 패킷을 **USB Serial JSON**으로 바꿔 Jetson에 전달하고, Jetson이 준 risk를 다시 **ESP-NOW**로 단말에 뿌립니다.

스스로 위험을 판단하지 않음 · 센서 없음

② Jetson Nano — 두뇌 (계산·판정) ★

거리·TTC·접근속도 계산, **AI(ONNX) 추론 실행**, zone 판정, **max 융합**, **최종 risk 0~3 결정**까지 전부 수행.

= 이 프로젝트의 실시간 두뇌, 내 담당의 핵심

RSU 노드의 물리 구성 (한 박스 안에 둘)

RSU ESP32 (무선 송수신) —USB Serial 115200bps— **Jetson Nano (거리·TTC·AI·max-risk 결정)** · 도로변 전원

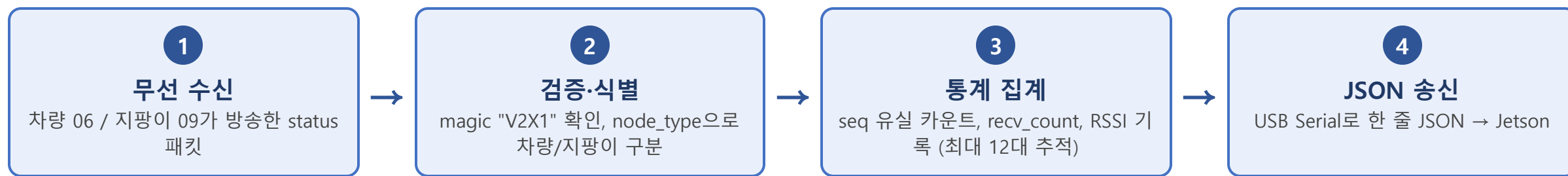
핵심 한 문장 (정정)

"RSU ESP32는 판단하지 않고 중계만 한다. 하지만 **RSU 노드 전체로 보면** 그 안의 **Jetson이 AI 추론과 최종 risk 판정을 수행한다.**" — 즉 **위험도 계산의 본진은 여기(RSU 현장)**이며, 서버나 SUMO가 아닙니다.

상행 경로 — 상태를 모아 Jetson으로

차량·지팡이의 무선 상태 패킷을 받아, 검증하고, 통계를 붙여, JSON 한 줄로 Jetson에 넘깁니다. (스케치

10_rsu_espnw_multi_jetson_bridge)



Jetson으로 나가는 JSON (예)

```
{
  "type": "vehicle", "node_id": 123,
  "seq": 118, "gps_valid": 1,
  "lat": 37.49, "lng": 126.95,
  "speed_mps": 8.3, "heading_deg": 90,
  "node_risk": 1, "recv_count": 118,
  "lost_count": 0, "rssi": -31,
  "src_mac": "AA:BB:CC:..."
}
```

필드가 말해주는 것

node_id / type — 누구인가(차량/지팡이)

lat·lng·speed·heading — 어디서 어떻게 움직이나

node_risk — 단말이 자체 계산한 1차 위험도

recv/lost/rssi — 통신 품질(패킷 손실·신호세기)

특징 차량·지팡이 status를 **모두** 받는 것이 10번(실사용 RSU).
08번은 차량만 받는 축소판입니다.

하행 경로 — 위험 경고를 되돌려준다

Jetson이 계산한 risk를 받아, 대상 단말에게 알맞은 형식으로 다시 무선 송신합니다. 대상이 지팡이면 진동이 울립니다.

Jetson에서 RSU로 들어오는 명령

```
// 전체 대상
{"risk":2}

// 특정 대상 지정
{"target_id":456,"src_id":123,"risk":3}
```

target_id = 경고를 받을 단말(지팡이)
src_id = 위험을 유발한 상대(차량)
risk = 최종 위험도 0~3

RSU의 중계 분기

대상이 **지팡이(CANE)**면 → MSG_RISK_ALERT(위험 경고 메시지)로 전송 → 지팡이가 진동·부저 출력.

대상이 **차량**이면 → MSG_RSU_REPLY로 전송 → 차량 알림.

node_id로 대상 슬롯을 찾고, 못 찾으면 risk_drop으로 버립니다.

정리

상행은 "정보 수집", 하행은 "경고 배달". RSU는 이 왕복 배달만 담당하고, 무엇이 위험한지는 Jetson이 정합니다.

위(상행)로 올라간 정보가 Jetson을 거쳐 아래(하행)로 경고가 되어 돌아옵니다.

차량 ESP32(06) —ESP-NOW status—
└─▶ RSU 브리지(10) —USB Serial JSON—▶ Jetson(위험도 계산)

지팡이 ESP32(09) —ESP-NOW status—

Jetson —USB {"target_id":지팡이,"src_id":차량,"risk":N}—▶ **RSU 브리지(10)**
├ 대상=지팡이 → MSG_RISK_ALERT →▶ 지팡이(09) → 진동·부저
└ 대상=차량 → MSG_RSU_REPLY →▶ 차량(06) → 알림

08 = 차량만 받는 단일 브리지(개발 초기). **10** = 차량+지팡이 모두 받는 **실사용 RSU**(최대 12대 추적).

10(RSU)을 **version 2 + distance** 필드로 올려야 09 지팡이가 RSU발 경고를 받습니다(8p 참고). 현재는 06↔09 직접 경로로 우회 동작.

RSU가 넘긴 뒤, Jetson에서 벌어지는 일

RSU가 준 JSON을 받아, Jetson의 파이썬 파이프라인(lux/)이 **6단계**를 거쳐 위험도를 만듭니다. (자세한 코드는 문서 2에서)

단계	파일	하는 일
3	parse_v2x.py	JSON 한 줄을 파싱해 17개 컬럼 표준 형식으로 정리 + 출처(source_mode) 표시
4	state_store.py	차량·지팡이의 최신 상태 를 보관, 위험 계산이 가능한 "쌍"인지 판정(신선도 0.5초)
5	sim_vehicle.py	실차가 없을 때 접근하는 가짜 차량 을 만들어 하위 단계를 테스트
6	kinematics.py	거리·접근속도·TTC·CPA 계산 + 칼만 필터로 GPS 잡음 평활
7	risk_scoring.py	점수표로 채점 → risk_level 0~3 (옆으로 스치는 차 억제 게이트 포함)
8	send_risk.py	risk를 RSU로 전송(신뢰 안 되는 GPS면 경보 억제, 변화 시 즉시/1초 주기)

흐름 parse → state → (sim) → kinematics → risk → send. 이 순서가 lux/ 파이프라인의 뼈대이며, 각 단계는 앞 단계의 출력을 입력으로 받습니다. 모든 단계에 단위 테스트(test_*.py)가 있습니다.

위험도는 어떻게 0~3이 되는가

실시간 경고는 **거리** 또는 **TTC** 임계값으로 즉시 판정합니다. "얼마나 가까운가"와 "얼마나 빨리 충돌 가능성이 생기는가"를 함께 봅니다.

risk	조건 (거리 또는 TTC)
RISK 3	거리 < 3m 또는 TTC < 2초
RISK 2	거리 < 6m 또는 TTC < 4초
RISK 1	거리 < 12m 또는 TTC < 6초
RISK 0	그 외 (안전)

TTC(충돌예상시간)

$rel_speed = (\text{이전거리} - \text{현재거리}) / \Delta t$

$TTC = \text{현재거리} / rel_speed$ (멀어지면 999)

Fire-first (선-발화)

지팡이는 **risk 2 이상이면 즉시 진동**합니다. 정밀 판단을 기다리지 않고 먼저 울립니다 — 놓치는 것보다 안전.

DCPA 게이트 (오경보 억제)

단순히 가까운 차가 아니라 **실제로 나에게 다가오는 차만** 걸러냅니다. 옆으로 스쳐 지나가는 차(CPA 거리가 충분)는 위험도를 낮춰 불필요한 진동을 줄입니다.

최종 통합

$final_risk = \max(rule, predicted, zone, fail_safe)$

여러 경로 중 **가장 높은 위험**을 채택 — 한쪽이 놓쳐도 다른 쪽이 잡습니다.

SUMO · RSU · 서버 — 어디서 무엇을 하나

셋 다 "risk"라는 말을 쓰지만 역할이 완전히 다릅니다. 이 표 하나로 경계가 정리됩니다.

장소	언제	여기서 하는 일	담당
SUMO / 로컬 PC	오프라인 (가끔)	기준을 만든다 — 점수표 정의, 가상 상황에 라벨(risk 0~3) 부여, Transformer 학습 → .onnx 산출 여기의 risk는 "정답지"이지 현장 경고가 아님	최민서
RSU 노드(Jetson) ★	실시간 (현장)	실제로 판정한다 — 거리·TTC 계산, zone 판정, ONNX 추론 실행 , max 융합, 최종 risk 0~3 결정 → 지팡이로 전송 경보를 좌우하는 유일한 실시간 판정 지점	나 (강현준)
서버 / 클라우드	사후 (누적)	보여준다 — 이미 결정된 risk를 받아 저장·집계, 도로 구간별 위험지도 표시 여기서 risk를 새로 만들지 않음(재계산 없음)	최민서(지도) + 나(업로드)

왜 실시간 판정이 RSU여야 하나

① **지연** — 차가 다가오는 판단을 서버 왕복(수백 ms~초)으로 하면 늦습니다. ② **단절** — 인터넷이 끊겨도 경고는 계속돼야 합니다. ③ **데이터 위치** — 원본 위치·속도가 도착하는 곳이 여기입니다.

왜 SUMO·서버는 안 되나

SUMO는 현장과 통신하지 않는 가상 실험이고(결과는 모델·라벨로만 전달), **서버**는 상시 연결·즉시성을 보장 못 합니다. 그래서 둘 다 실시간 판정에는 부적합합니다.

결론 정의는 SUMO(최민서), 판정은 RSU/Jetson(나), 표시는 서버. 겹치는 게 아니라 정의 → 실행 → 기록으로 이어지는 한 줄입니다.

max 융합과 risk 레벨화 — 전부 Jetson에서

네 갈래 판단이 **Jetson** 한 곳으로 모여 max로 합쳐지고, 거기서 최종 등급이 정해집니다. (근거: lux/fusion.py, lux/risk_scoring.py)

Jetson 안으로 모이는 4갈래

- **rule** — 거리·TTC 규칙 (Jetson 계산)
- **predicted** — ONNX 모델 추론 (모델은 최민서 제작, 실행은 Jetson)
- **zone** — 위험구역 판정 (구역 정의는 최민서, 판정은 Jetson)
- **fail-safe** — 통신 품질 저하 시 보수적 유지

합치는 규칙 = 안전 우선 max

```
final = max(rule, predicted, zone, fail_safe)
```

한 갈래가 놓쳐도 다른 갈래가 잡습니다.

레벨화(0~3)는 두 곳에서, 목적이 다름

① 라벨용 (오프라인·최민서)

SUMO 데이터에 정답 라벨을 붙이려고 점수표로 0~3 산출. 학습 용이며 현장 경고와 무관.

② 실시간용 (온라인·나) ★

현장 데이터로 점수·임계값을 계산해 실제 경고 등급 0~3 결정. 지팡이를 울리는 건 이쪽뿐.

⚠ 지금 남은 문제 — 정의가 두 벌

A 점수표(≥70/45/20)와 **B 거리임계**(<3/6/10m)가 공존해 같은 상황이 다른 등급이 됩니다. **Transformer가 A로 학습됐으므로 A로 통일**이 필요합니다.

서버와는 어떻게 연결하는가

현재는 로컬 CSV 저장이 기본이고, 서버/클라우드 연동은 향후 확장으로 설계되어 있습니다. 연동 규격은 통합 계약서에 초안이 있습니다.

지금 (구현·검증 완료)

로컬 CSV 로깅

Jetson이 각 단계 결과를 파일로 남깁니다.

step6_kinematics_log.csv (운동학)

step7_risk_log.csv (위험도)

step8_risk_tx_log.csv (전송)

final_risk_log.csv (seq·rssi·수신간격)

즉, 서버가 없어도 시스템은 완결적으로 돌아가며, 로그는 나중에 위험지도·통계로 재활용됩니다.

향후 (확장 설계)

클라우드 위험지도

위험 이벤트(시간·위치·id·risk_level·reason)를 서버 DB로 올려 도로 구간별 누적 위험지도·대시보드로 시각화. 통합 계약서에 POST /api/events 업로드 계약 초안 존재.

DB 테이블(설계)

cane_state · vehicle_state(최신 상태) → risk_log(위험 판단) → event_log(경고 이력), zone(위험 구역). PK/FK 관계로 이력 저장.

SUMO는 왜 쓰는가

실제 도로에서 위험한 차량 접근을 수백 번 재현하는 건 위험하고 어렵습니다. 그래서 **SUMO(오픈소스 교통 시뮬레이터)**로 안전하게 대량의 위험 상황 데이터를 만듭니다.

현실 재현

OpenStreetMap 실제 도로(캠퍼스 주변)를 가져와 교차로·횡단보도·주차장 출구를 그대로 반영.

시나리오 생성

정면 접근·측면 사각지대·다중 차량 등 위험 상황을 원하는 만큼 반복 생성.

AI 학습 자료

차량·보행자의 위치·속도·거리를 정확한 수치로 뽑아 **AI 학습 데이터**로 사용.

어떤 위험 상황을 만드는가 (시나리오)

S01 정면 접근(High) · **S02 측면 사각지대(High)** · **S03 안전 통과(Low)** · **S04 횡단보도 접근(Medium)** · **S05 다중 차량 접근(High)**. 각 시나리오는 거리·TTC·상대속도로 위험도가 다르게 나오도록 설계되어 있습니다.

참고 SUMO 원본 프로젝트(.net.xml / .rou.xml / preprocess.py)는 이 저장소가 아닌 별도 백업(SUMO_project_backup.lnk)에 있습니다. 저장소에는 **결과 CSV**가 들어옵니다.

SUMO → 데이터셋 → AI 학습

시뮬레이션 원시 데이터가 여러 스크립트를 거쳐 **AI 학습용 최종 데이터셋**이 됩니다. (스크립트 상세는 문서 2)

순서	산출물 / 스크립트	하는 일
1	SUMO 시뮬레이션	차량·보행자 궤적을 FCD(XML)로 출력 → CSV 변환
2	feature.csv (9컬럼)	ts_ms, ped_x/y, veh_x/y, 속도, distance, rel_speed
3	run_scenario.py	시나리오별로 rule 위험도 + 이벤트 분석 자동 실행(오케스트레이터)
4	event_analyzer.py	zone 판정 + risk 점수/등급 + event_type 부여 → event_result.csv (22컬럼)
5	merge_dataset.py	모든 시나리오 결과를 세로로 합침 → master_dataset.csv
6	merge_labels.py	피처+라벨을 키로 검증 병합 → labeled_master_dataset.csv (학습 입력)
7	train_transformer.py	Transformer 학습 → .pt / scaler.json / report
8	export_onnx.py	.pt → .onnx (엣지 경량 추론용)

핵심 SUMO가 만든 상황을 **rule(규칙)**로 **채점**해 정답 라벨(risk_level)을 붙이고, 그 라벨을 **AI가 학습**합니다. 즉 AI는 rule을 **대체**하는 게 아니라 rule의 산출물을 **학습·보강**합니다.

AI 모델 — Transformer 위험도 분류기

최근 10프레임의 움직임을 보고 현재 위험 등급(0~3)을 분류하는 Transformer입니다. 학습 후 ONNX로 변환해 Jetson에서 가볍게 추론합니다.

입력 — 11개 피쳐 × 10프레임

ped_x, ped_y, veh_x, veh_y,
ped_speed, veh_speed,
distance_m, rel_speed,
ttc, risk_score, zone_base_risk

→ StandardScaler 정규화 → 시퀀스 [batch,10,11]

주의 — 뒤 3개는 rule 산출물

ttc·risk_score·zone_base_risk는 센서 원값이 아니라 **rule**이 먼저 계산한 값입니다. AI를 돌리려면 rule을 먼저 계산해 피쳐로 넣어야 합니다.

구조 & 출력

Linear(11→64) → TransformerEncoder(d=64, head=4, layer=2)
→ 마지막 프레임 → 분류(4클래스).

출력 [batch,4] logits → argmax = risk_level.

99.3%

테스트 정확도

4

클래스
(정상/주의/경고/위험)

10

시퀀스 길이
(프레임)

데이터 불균형 주의: 'Near Miss' 클래스 샘플이 6개로 극소수 → 실사용 신뢰도 낮음. 데이터 보강 필요.

데이터가 자라나는 순서 (CSV 계보)

SUMO 원시 9컬럼에서 시작해 라벨이 붙으며 컬럼이 늘어납니다. 어떤 CSV가 무엇인지 알면 데이터 흐름이 보입니다.

파일	컬럼	내용
scenario_00X/feature.csv	9	SUMO 원본 피쳐(위치·속도·거리·상대속도)
scenario_00X/risk_result.csv	11	feature + ttc + risk_level (Jetson rule 결과)
scenario_00X/event_result.csv	22	feature + risk 점수/등급 + zone + event_type (라벨링 완료)
master_dataset.csv	24	6개 시나리오 event_result 전체 병합(약 7.9만 행)
labeled_master_dataset.csv	23	AI 학습 입력 (피쳐+라벨 정합, 약 1.26만 행)
feature_with_zone.csv	15	feature + zone만(위험도 없음, 중간 산출물)

⚠ 재현성 주의

master_dataset.csv(약 7.9만 행)와 labeled_master_dataset.csv(약 1.26만 행)의 행 수가 다릅니다. merge_labels.py는 두 입력의 행 수가 같아야 하므로, 현재 학습 데이터는 **과거 단일 시나리오 시점**에 만들어진 것으로 보입니다. 재현 시 경로/버전 정합을 맞춰야 합니다.

지금 어디까지 왔고, 무엇이 남았나

완료 (구현·검증)

- ESP-NOW 송수신, RSU 브리지, Jetson UART 수신 (약 9.88Hz)
- lux/ 파이프라인 1~8단계 + 단위 테스트 다수
- 거리·TTC 규칙 기반 실시간 위험도 0~3
- Fire-first 즉시 경고, DCPA 오경보 억제
- SUMO 데이터셋 → Transformer 학습 → ONNX 변환

남은 일 (확장·정합)

- **9단계**: 가짜 차량 → 실차 ESP32로 교체 (6~8단계 코드 유지)
- RSU(10)를 **version 2 + distance**로 올려 09와 정합
- AI(ONNX) 추론을 실시간 파이프라인에 연결
- 좌표계 통일(GPS 위경도 ↔ SUMO 로컬 미터)
- 서버/클라우드 위험지도 연동

인수인계 핵심 주의 5가지

① 프로토콜 계열 A/B 혼재 ② 계열 B 내 v1/v2 차이(RSU↔지팡이) ③ 위험도 정의 2벌(점수표 vs 거리임계) ④ 좌표계 3종 미통일 ⑤ 학습 데이터 행수 불일치. — 자세한 내용은 각 페이지의 주황/빨강 카드 참고.

어디에 무엇이 있나

처음 저장소를 열었을 때 길을 잃지 않도록, 폴더별 역할을 정리했습니다.

arduino/

ESP32 펌웨어 01~10 (차량·지팡이·RSU). 각 폴더에 README. **RSU=08,10**
/ 차량=06 / 지팡이=09.

scripts/

SUMO 결과 처리(run_scenario, event_analyzer, risk_calculator,
zone_detector, merge_dataset).

dataset/

시나리오별 feature/risk/event CSV + 통합 master/labeled 데이터셋.

docs/integration_contract.md

세 모듈을 붙일 때의 규격 합의서. 좌표계·feature·위험도 정의 정합. 반드시 정독.

lux/

Jetson 위험도 **파이썬 파이프라인**(parse→state→kinematics→risk→send)
+ 테스트. ← 내 RSU 담당 핵심.

AI_Model/transformer/

Transformer 학습·ONNX 변환·검증 + 학습된 모델(.pt/.onnx/scaler.json).

python/jetson_run/

Jetson 실행용 rule 추론(main.py, risk_inference.py) + 9컬럼 입력.

lux/v2x_session_summary_next_steps.md

단계별 진행·결정 기록(마스터 인수인계 메모). lux/ 작업의 1차 출처.

환영합니다 · 이제 시작할 차례

큰 그림을 잡았다면, 다음은 코드 설명서로

이 문서로 무엇을 만드는지, 9단계가 어떻게 이어지는지, RSU가 무슨 일을 하는지, 서버·SUMO가 어떻게 연동되는지를 잡았습니다.

문서 2 · 코드 설명서에서는 파이썬 파일 하나하나가 실제로 무슨 일을 하는지, RSU 파이프라인의 어느 단계이고 다음으로 어떻게 넘어가는지 순서대로 설명합니다.